

A
PROJECT BASED LEARNING REPORT
ON
“PYTHON BASED NETWORK ASSESMENT TOOLKIT”
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING (CYBERSECURITY)
BY

RANGISETTI SREEKAR – 24N81A6235

YASH KUMAR LIDHORYA - 24N81A6233

GANGULA PRAHALAD – 24N81A6212

THATI KIRAN TEJA - 24N81A6209

BHANU PRAKASH - 24N81A6210

Under the Guidance of

Mrs. A.Sandhya Rani

Assistant Professor

Department of CSE (CYBER SECURITY)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CS)

SPHOORTHY ENGINEERING COLLEGE

Nadergul (Vill.), Near L.B. Nagar, Hyderabad, Telangana – 501510.

[2024-2028]

SPHOORTHY ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CS)



CERTIFICATE

This is to certify that this Project Based Learning Report entitled PYTHON BASED NETWORK ASSESSMENT TOOLKIT is a Bonafide work carried out by Rangiseti Sreekar(24N81A6235), Yash Kumar (24N81A6233), Kiran Teja(24N81A6209), Gangula Prahalad(24N81A6212), Bhanu Prakash(24N81A6210) in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology in Computer Science and Engineering (CS)** from **Sphoorthy Engineering College**, affiliated to Jawaharlal Nehru Technological University Hyderabad, during the Academic Year 2025-26 under our guidance and supervision.

The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma.

Internal Guide

Mrs. A.Sandhya Rani
Assistant Professor
Department of CSE (CS) SPHN

Head of the Department

DECLARATION

We the undersigned, declare that the Project Based Learning Report “PYTHON BASED NETWORK ASSESSMENT TOOLKIT” carried out at “SPHOORTHY ENGINEERING COLLEGE” is original and is being submitted to the Department of COMPUTER SCIENCE AND ENGINEERING (CYBERSECURITY), Sphoorthy Engineering College, Hyderabad towards partial fulfilment for the award of degree of Bachelor of Technology.

We declare that the result embodied in the the Project Based Learning Report has not been submitted to any other University or Institute for the award of any Degree or Diploma.

Date:05/06/2026

Place: Hyderabad

RANGISETTI SREEKAR – 24N81A6235

YASH KUMAR LIDHORIYA - 24N81A6233

GANGULA PRAHALAD – 24N81A6212

THATI KIRAN TEJA - 24N81A6209

BHANU PRAKASH - 24N81A6210

ACKNOWLEDGEMENT

We sincerely thank our Project Guide Mrs. A. Sandhya Rani, Asst. Professor, Department of Computer Science & Engineering (CYBERSECURITY), Sphoorthy Engineering College, for her inspiring guidance, consistent encouragement, constructive criticism and helpful suggestions during the entire course of my project work.

We express our sincere thanks to Mrs. P. Sandhya Rani, Professor & Head of the Department, Department of Computer Science & Engineering (CS), Sphoorthy Engineering College, Nadergul (V), Balapur (M), Ranga Reddy (D) for their encouragement which helped us to complete the Project work.

We deem it a great privilege to express our profound gratitude and sincere thanks to Mr. S. Chalama Reddy, Chairman, Mr. S. Jagan Mohan Reddy, Secretary, Dr. K. Srinivas Rao, Principal, Sphoorthy Engineering College, Nadergul (V), Balapur (M), Ranga Reddy (D), for their moral support and help in the completion of the project work.

We express our heartfelt thanks to Professors, Associate Professors, Assistant Professor and other professional non-teaching staff of Department of Computer Science and Engineering (CS), Sphoorthy Engineering College, Nadergul (V), Balapur (M), Ranga Reddy (D) for providing the necessary information pertaining to our project work.

BATCH: 2024 – 2028

RANGISETTI SREEKAR – 24N81A6235

YASH KUMAR LIDHORYA - 24N81A6233

GANGULA PRAHALAD – 24N81A6212

THATI KIRAN TEJA - 24N81A6209

BHANU PRAKASH - 24N81A6210

ABSTRACT

This project presents the design and development of a Python-based port scanner with GUI, service identification, and banner grabbing for local network security assessment. The system is developed to identify open TCP ports running on a target machine and display associated services and banners in a user-friendly graphical interface.

The scanner supports multiple scan modes including Quick Scan, Common Ports Scan, Custom Scan, and Full Scan. It uses multithreading to improve scanning speed and efficiency. The project also performs dynamic service detection using Python socket programming and attempts banner grabbing to collect additional service information.

The system helps users understand open ports, active services, and potential network exposure within authorized local environments. The project demonstrates basic cybersecurity concepts such as TCP scanning, service identification, and network reconnaissance in an educational and defensive context.

The proposed toolkit architecture is designed to support multiple network security assessment modules including Port Scanning, Service Identification, Banner Grabbing, DNS Lookup, Host Discovery, and Risk Analysis. In the current implementation, the Port Scanner, Service Detection, Banner Grabbing, and Export functionalities have been successfully developed and tested. Additional modules are proposed for future enhancement of the toolkit.

RANGISETTI SREEKAR – 24N81A6235

YASH KUMAR LIDHORYA - 24N81A6233

GANGULA PRAHALAD – 24N81A6212

THATI KIRAN TEJA - 24N81A6209

BHANU PRAKASH - 24N81A6210

INDEX

Contents			Page No.
Abstract			I
Index			II
List of Figures			III
CHAPTERS			
1.	Introduction		1
	1.1	Problem Statement	1
	1.2.	Objective	1
	1.3.	Existing System	1
	1.4.	Proposed System	2
	1.5.	Scope	2
	1.6.	Software & Hardware Requirements	3
2.	Literature Survey		4
	2.1.	Port Scanning	4
	2.2.	TCP/IP Networking	4
	2.3.	Socket Programming	5
	2.4	Banner Grabbing	
3.	System Design		6
	3.1.	System Architecture	6
	3.2.	System Flow	6
	3.3.	Module Description	7
	3.4.	UML Diagram	8
4.	Implementation		13
	4.1.	Environmental Setup	13
	4.2.	Development Tools	13
	4.3.	Implementation of Modules	15
5.	Evaluation		16
	5.1.	Test Cases	16
	5.2.	Results	17

	5.3.	Advantages	17
	5.4	Limitations	23
6.	Conclusion and Future Enhancement		24
	References		26
	Appendix		27

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.2.1	Use Case Diagram	8
3.2.2	Class Diagram	9
3.2.3	Sequence Diagram	9
3.2.4	Activity Diagram	10
5.3.1	EAPCET Rank Entry Page	17
5.3.2	Technologies Exploration Dashboard	18
5.3.3	Skill&Interests Enhancement Test	20
5.3.4	Career Path Guide	21
5.3.5	Domian Benefits Information Page	23

CHAPTER-1

INTRODUCTION

1.1 PROBLEM STATEMENT

Computer systems may contain multiple open ports running different services. Unnecessary or exposed ports can increase security risks and system vulnerability. Manually identifying open ports is difficult and time-consuming. Therefore, there is a need for a simple tool that can scan target systems, identify open ports, and display related service information.

1.2 OBJECTIVE

- Detect open TCP ports
- Identify services running on ports
- Perform banner grabbing
- Provide GUI-based interaction
- Support multiple scan modes
- Improve scan speed using multithreading

1.3 EXISTING SYSTEM

Existing tools like Nmap provide advanced scanning features. However, these tools may be complex for beginners and difficult to understand in educational environments.

Drawbacks:

- Complex interface
- Advanced configuration required
- Difficult for beginners
- Large learning curve

1.4 PROPOSED SYSTEM

The proposed system is a Python-based GUI port scanner that performs TCP connect scanning, service identification, and banner grabbing. The system provides multiple scan modes and real-time progress tracking.

1.5 Scope

- Educational cybersecurity learning
- Local network assessment
- Understanding open ports and services
- Demonstration of socket programming

1.6 SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

- **Operating System:** Windows 10 / Windows 11/Ubuntu 24+
- **Programming Language:** Python,
- **Compiler / IDE:** Visual Studio Code
- **Libraries:** tkinter

Hardware Requirements

- **RAM:** 4 GB (Minimum), 8 GB Recommended
- **Storage:** 50 GB Free Disk Space (Minimum)
- **System Type:** 64-bit Operating System, x64-based Processor
- **Processor:** Intel Core i3 / Intel Core i5 or equivalent AMD processor (Minimum 2.0 GHz)

CHAPTER-2

LITERATURE SURVEY

2.1 Port Scanning

Port scanning is a technique used to identify open ports and active services running on a target system. Port scanning is a network reconnaissance technique used to identify open ports and active services running on a target system. Every network service communicates through specific port numbers, and open ports indicate that a particular service is listening for incoming connections. Port scanning helps administrators and security professionals understand which services are exposed to a network and whether unnecessary services are running.

2.2 TCP/IP Networking

TCP/IP (Transmission Control Protocol / Internet Protocol) is the fundamental communication protocol suite used for communication across computer networks and the internet. It provides a standardized method for transmitting data between devices connected through networks.

The TCP/IP model consists of multiple layers responsible for different networking operations such as addressing, routing, transportation, and application-level communication. TCP is a connection-oriented protocol that establishes reliable communication between devices before transferring data. It ensures packet delivery, sequencing, and error checking.

2.3 Socket Programming

Socket programming is a method of enabling communication between systems over a network using software endpoints called sockets. A socket acts as an interface between the application layer and the transport layer of networking protocols.

2.4 Banner Grabbing

Banner grabbing is a technique used to collect information from services running on open ports. Many network services respond with identifying information immediately after a connection is established. This response is known as a banner.

2.5 Multithreading in Port Scanning

Multithreading is a programming technique where multiple threads execute tasks simultaneously within a program. In port scanning, scanning ports sequentially can be slow because each connection attempt requires waiting for a response or timeout.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

Simple Architecture Flow:

The system architecture of the port scanner is shown in the figure below. The user provides the target IP and scan options through the GUI. The scanner engine performs port scanning using socket programming, detects services, grabs banners and displays the results.

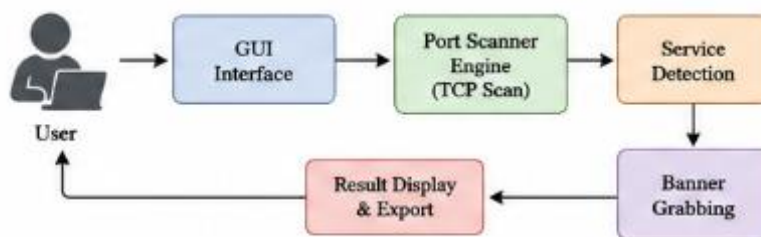


Fig 3.1 System Architecture

3.2 SYSTEM FLOW

1. User enters target IP
2. Selects scan mode
3. Scanner checks ports
4. Open ports identified
5. Service detection performed
6. Banner grabbing attempted
7. Results displayed

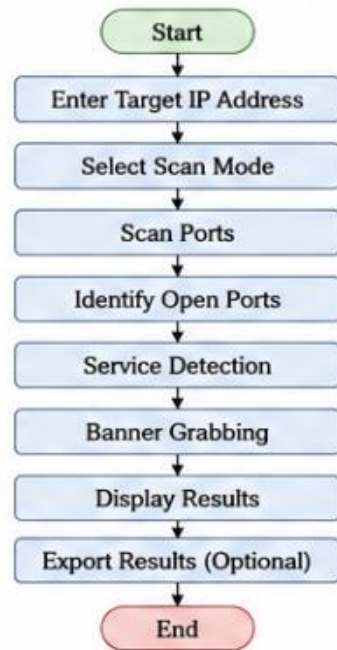


Fig 3.2 System Flow Diagram

3.3 MODULE DESCRIPTION

User Interface Module

Provides GUI interaction.

Port Scanning Module

Performs TCP connect scanning.

Service Detection Module

Identifies service names dynamically.

Banner Grabbing Module

Collects service banners.

Export Module

Exports scan results.

3.4 UML DIAGRAM

1. Use Case Diagram

Use Cases:

- Enter Target IP Address
- Select Scan Mode
- Start Scan
- View Scan Results
- Export Results

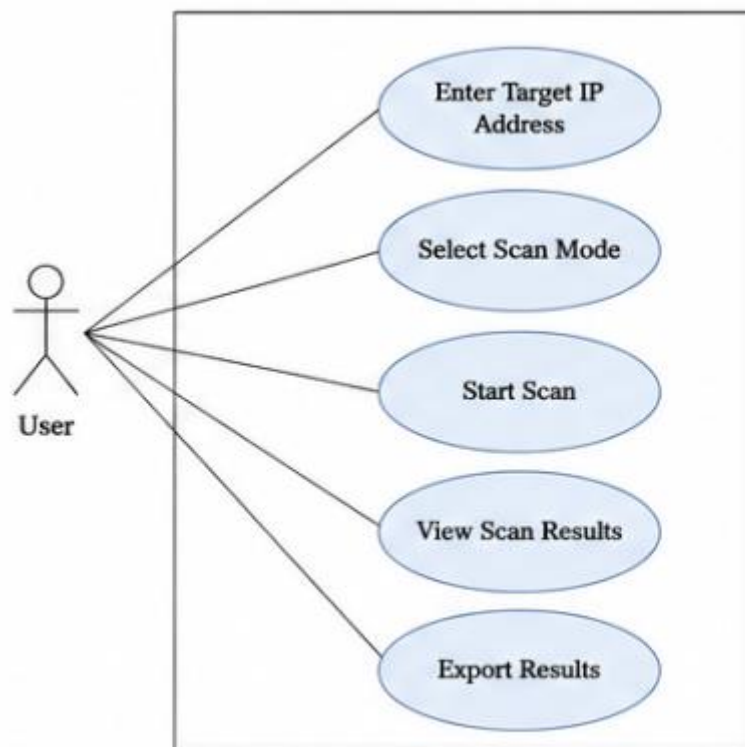
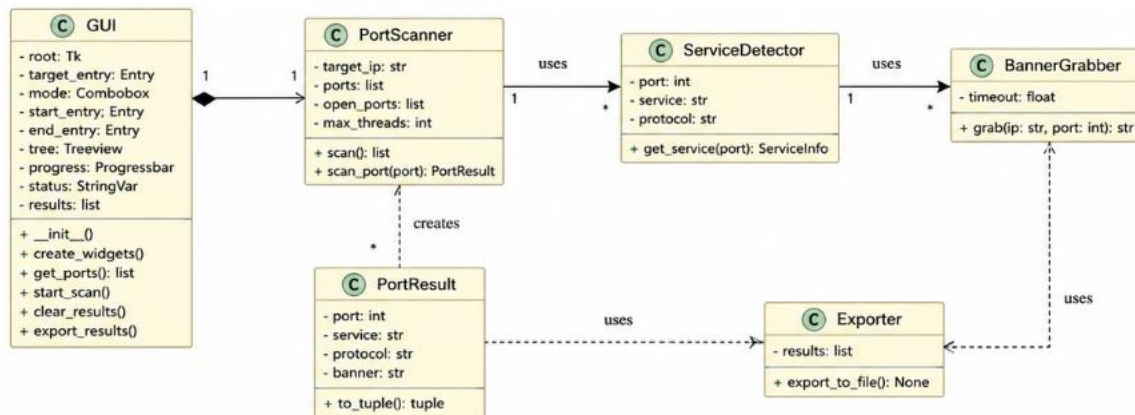


Fig 3.3 Use Case Diagram

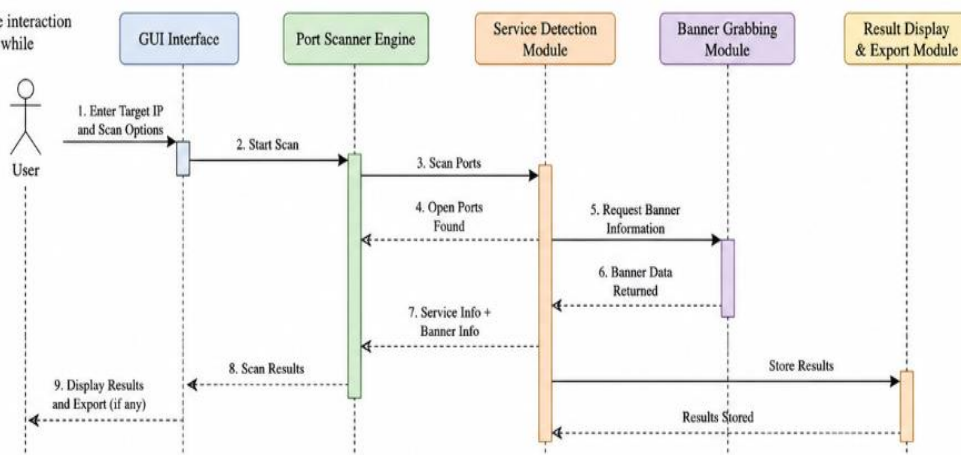
2. Class Diagram

The Class Diagram Represents the structure of the system and the relationship between classes



3. Sequence Diagram

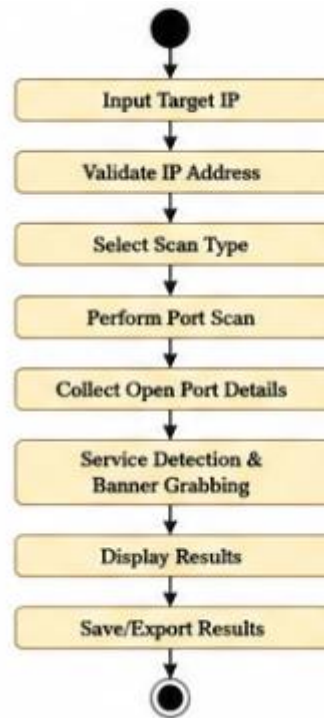
The sequence diagram shows the interaction between the user and the system while performing a port scan.



4. Activity Diagram

- **Start:** User launches the Port Scanner application.
- **Enter Target IP:** User enters the target IP address or hostname.
- **Select Scan Mode:** User selects Quick Scan, Common Scan, Custom Scan, or Full Scan.
- **Validate Input:** System validates the entered IP address and port range.
- **Start Port Scanning:** Scanner begins TCP port scanning using multithreading.
- **Check Open Ports:** System checks whether target ports are open or closed.
- **Service Detection:** Open ports are matched with associated services.

- **Banner Grabbing:** System retrieves available banner information from open ports.
- **Display Results:** Scan results are displayed in the GUI table with port, service, protocol, and banner details.
- **Export Results:** User can save scan results into a text file.
- **End:** Scanning process completed successfully.



CHAPTER-4

IMPLEMENTATION

4.1 ENVIRONMENT SETUP

The environment setup for the Python-Based Port Scanner involves installing the required software, libraries, and tools needed for the development and execution of the system. The project uses Ubuntu Linux / Windows as the operating system with Python as the primary programming language and Visual Studio Code as the development environment. Required libraries such as Tkinter, Socket, and Concurrent Futures are used for GUI development, network communication, and multithreading functionality.

The system is developed to perform TCP port scanning, service identification, banner grabbing, and result display using a graphical interface. After installing all required components, the project files are configured and executed successfully for local network security assessment.

4.1.1 Development Environment

Operating System: Ubuntu Linux / Windows 10 / Windows 11

Programming Language: Python

Development Tools / IDE: Visual Studio Code

Libraries: Tkinter, Socket, Concurrent Futures, Threading

GUI Framework: Tkinter

Networking Technology: TCP/IP Socket Programming

Web Browser: Google Chrome / Mozilla Firefox (Optional)

Version Control: Git (Optional)

Setup Steps:

Install Ubuntu Linux / Windows operating system.

Install Python programming language.

Install Visual Studio Code IDE.

Install Tkinter and required Python libraries.

Create project files and configure the GUI environment.

Implement socket programming for TCP port scanning.

Implement service detection and banner grabbing functionality.

Run the application and test scanning functionality.

4.2 IMPLEMENTATION OF MODULES

GUI Module

The GUI Module is implemented to provide user interaction with the system. It allows users to enter target IP addresses, select scan modes, start scanning, clear results, and export scan reports through a graphical interface.

Port Scanning Module

The Port Scanning Module is implemented to perform TCP connect scanning on target systems. It checks selected ports and identifies whether the ports are open or closed using socket communication.

Service Detection Module

The Service Detection Module is implemented to identify services associated with open ports. It uses Python socket functions and predefined mappings to display common services such as HTTP, HTTPS, SSH, and MySQL.

Banner Grabbing Module

The Banner Grabbing Module is implemented to collect response information from services running on open ports. It retrieves available banners and displays readable service information whenever possible.

Multithreading Module

The Multithreading Module is implemented to improve scanning speed and efficiency. Multiple ports are scanned simultaneously using ThreadPoolExecutor to reduce overall scan time.

Export Module

The Export Module is implemented to save scan results into text files for future reference and reporting purposes.

4.3 INTEGRATION AND DEVELOPMENT

Development

Creation of the Python-Based Port Scanner using Python socket programming, Tkinter GUI, and multithreading techniques. Modules such as port scanning, service detection, banner grabbing, and result exporting are developed and integrated into the system.

Integration

Combining all modules into a single GUI-based application by integrating user interface components, scanning logic, service detection, banner grabbing, and export functionality.

Testing and Refinement

Testing the system using localhost and local network IP addresses to identify errors, improve performance, fix bugs, and enhance scanning reliability and GUI responsiveness.

Deployment

Running the completed application on Ubuntu Linux / Windows systems for educational and local network security assessment purposes with proper authorization and safe usage practices.

CHAPTER-5

EVALUATION

5.1 MODULES

Evaluation of the Python-Based Port Scanner is carried out to measure the performance, usability, scanning efficiency, reliability, and accuracy of the system. The application is evaluated based on scan speed, service detection accuracy, GUI responsiveness, banner grabbing functionality, and overall user experience. The system is tested using localhost and local network targets to ensure proper scanning behavior and result generation.

GUI Module (User Interface Module)

The GUI Module manages user interaction with the Port Scanner system. It allows users to enter target IP addresses, select scanning modes, start or clear scans, and view scan results in a graphical table format. The module improves usability by providing a simple and user-friendly interface.

Port Scanning Module

The Port Scanning Module performs TCP connect scanning on selected target ports. It attempts socket connections to identify whether ports are open or closed. This module forms the core functionality of the system and supports Quick Scan, Common Scan, Custom Scan, and Full Scan operations.

Service Detection Module

The Service Detection Module identifies services associated with open ports using Python socket functions and predefined service mappings. It displays standard services such as HTTP, HTTPS, SSH, MySQL, and other known protocols.

Banner Grabbing Module

The Banner Grabbing Module collects response information from services running on open ports. It retrieves readable service banners whenever available and displays them in the GUI table. This helps users understand additional information about the detected services.

Multithreading Module

The Multithreading Module improves scanning performance by scanning multiple ports simultaneously. It uses Python's ThreadPoolExecutor to reduce total scanning time and improve efficiency during large scans such as Full Scan mode.

Export Module

The Export Module allows users to save scan results into text files for reporting and future reference. The exported data includes port number, service name, protocol type, and banner information.

5.2 EVALUATION METRICS

- Identify scanning performance and response time.
- Measure open port detection accuracy.
- Evaluate service identification efficiency.
- Check GUI responsiveness during scanning.
- Measure multithreading performance improvement.
- Analyze banner grabbing functionality.
- Verify export functionality and result accuracy.
- Test system stability during full port scans.

5.3 TEST CASES

Requirement Analysis

Understand all modules of the system:

- GUI Interface
- Port Scanning
- Service Detection
- Banner Grabbing
- Export Functionality
- Multithreading

Test Planning

- Decide what to test (modules & features)
- Select testing types:
 - Functional testing
 - Performance testing
 - GUI testing
 - Validation testing
- Prepare test environment (localhost / local network system)

Test Case Design

- Write test cases for each module
- Include:
 - Valid IP address testing
 - Invalid IP address testing
 - Quick Scan testing
 - Custom Range Scan testing
 - Full Scan testing
 - Banner Grabbing testing
- Define expected outputs for each case

Test Environment Setup

- Install Python and Tkinter libraries
- Configure GUI environment
- Run project files
- Ensure localhost and target systems are reachable

Final Testing

- Run all modules together
- Perform localhost and full scan testing
- Verify GUI responsiveness and result accuracy
- Check export functionality and performance

Test Closure

- Prepare final testing report
- Document pass/fail results
- Confirm project is stable and ready for demonstration and deployment

The main objective of this module is to:

- Collect the student's EAPCET rank.
- Validate the entered rank.
- Provide secure and smooth navigation to the next stage.
- Generate personalized recommendations based on the entered rank.

5.4 RESULTS

The Results Module of the Python-Based Port Scanner system displays the final output generated after scanning the target system. The system provides information about open ports, associated services, protocol types, and available banner details through the graphical user interface.

Results Obtained

- Successfully performed TCP port scanning on localhost and local network systems.
- Detected open ports running active network services.
- Identified associated services such as HTTP, HTTPS, SSH, and MySQL.
- Retrieved available banner information from open ports using banner grabbing.
- Displayed scan results in a GUI table with port number, service, protocol, and banner details.
- Implemented multiple scan modes including Quick Scan, Common Scan, Custom Scan, and Full Scan.
- Improved scanning performance using multithreading techniques.
- Successfully exported scan results into text files for reporting and future reference.

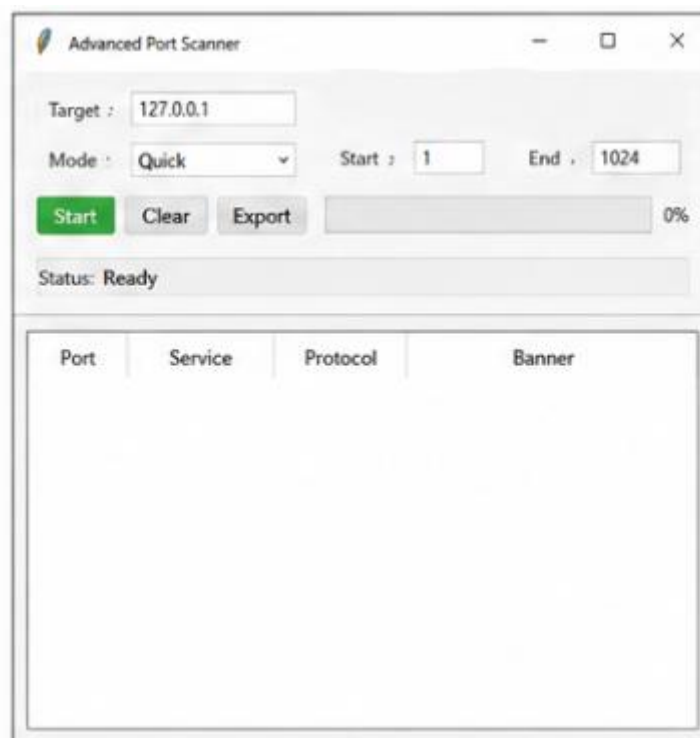


Fig 5.1 Main GUI Interface

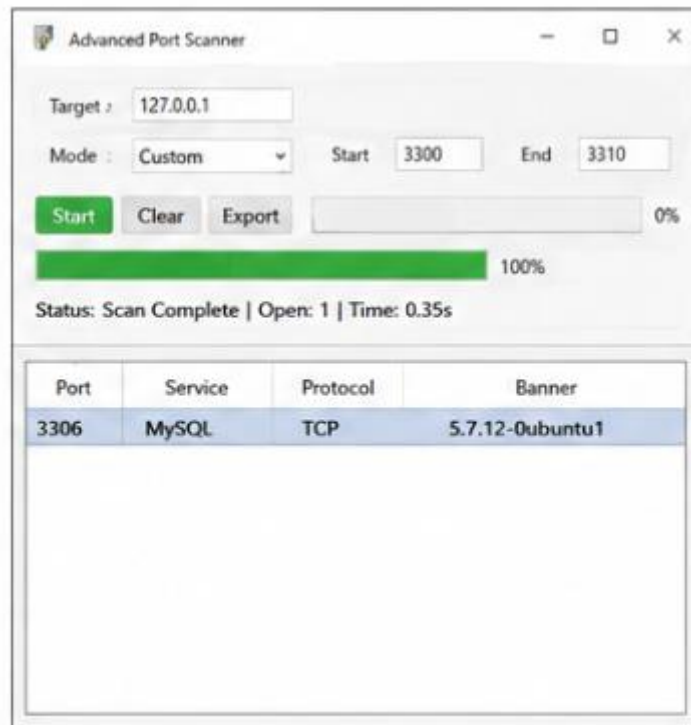


Fig 5.2 Scan Results (Port 3306)

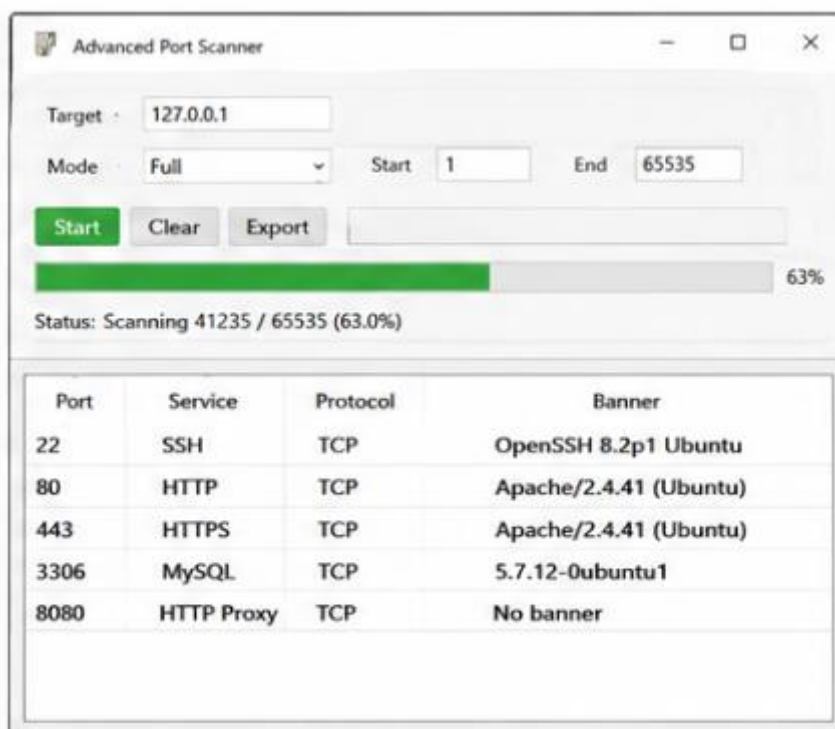


Fig 5.3 Full Scan Progress

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT

CONCLUSION:

The Python-Based Port Scanner with GUI, Service Identification, and Banner Grabbing was successfully designed and developed for local network security assessment. The system performs TCP port scanning to identify open ports and active services running on a target machine. The project provides a graphical user interface that allows users to perform scanning operations easily and efficiently.

The scanner supports multiple scan modes such as Quick Scan, Common Scan, Custom Scan, and Full Scan. The system also performs service identification and banner grabbing to collect additional information about detected services. Multithreading techniques were implemented to improve scanning speed and overall performance.

The project demonstrates important cybersecurity concepts such as TCP/IP networking, socket programming, port scanning, service detection, and banner grabbing. The developed system is useful for educational purposes and basic local network analysis within authorized environments.

Overall, the project successfully achieved its objectives by providing a functional, user-friendly, and efficient network scanning tool.

FUTURE ENHANCEMENTS:

- The current system can be further improved by adding advanced features and functionalities in future versions. Possible future enhancements include:
- Implementation of UDP port scanning for wider protocol support.
- Addition of Operating System detection techniques.
- Advanced service fingerprinting and version detection.
- Improved banner grabbing for more accurate service information.
- Generation of PDF and CSV scan reports.
- Implementation of real-time network host discovery.
- Addition of dark mode and enhanced GUI customization.
- Integration of vulnerability assessment features for security analysis.
- Support for remote network scanning and subnet scanning.
- Improvement of scanning optimization and performance efficiency.
- DNS Lookup Module
- Host Discovery Module
- Risk Analysis Module

- UDP Scanning
- OS Detection

• REFERENCES

- Python Software Foundation, *Python Official Documentation*, Available at: [Python Official Documentation](#)
- Python Software Foundation, *Socket Programming in Python*, Available at: [Python Socket Documentation](#)
- Python Software Foundation, *Tkinter GUI Documentation*, Available at: [Tkinter Documentation](#)
- Gordon Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*, Insecure.Com LLC.
- Andrew S. Tanenbaum, *Computer Networks*, Pearson Education.
- Behrouz A. Forouzan, *Data Communications and Networking*, McGraw-Hill Education.
- W. Richard Stevens, *TCP/IP Illustrated Volume 1: The Protocols*, Addison-Wesley Professional.
- Cisco Networking Academy, *Introduction to Cybersecurity*, Cisco Press.
- Python Software Foundation, *Concurrent Futures Documentation*, Available at: [Concurrent Futures Documentation](#)
- Kurose and Ross, *Computer Networking: A Top-Down Approach*, Pearson Education.

APPENDIX AND SAMPLE CODE

A.1 SOURCE CODE DESCRIPTION

The developed system uses Python programming language and consists of multiple modules integrated into a GUI-based application. The source code includes functionalities such as:

- GUI creation using Tkinter
- TCP socket programming
- Port scanning implementation
- Service detection
- Banner grabbing
- Multithreading using ThreadPoolExecutor
- Scan result exporting functionality

SAMPLE CODE:

```
import socket

import threading

import tkinter as tk

from tkinter import ttk, messagebox

from concurrent.futures import ThreadPoolExecutor

def scan_port(ip, port):

    try:

        s = socket.socket(

            socket.AF_INET,

            socket.SOCK_STREAM
```

```

    )

    s.settimeout(0.5)

    result = s.connect_ex((ip, port))

    if result == 0:

        return port

    s.close()
except:
    pass

return None

target = "127.0.0.1"
for port in range(1, 1025):
    result = scan_port(
        target,
        port
    )

    if result:
        print(
            "Open Port:",
            result
        )

```

